

Habitat — Source Distribution Notes

1.0 Release | CSRocks Inc.

1. Source Code Repository

The complete source code for Habitat is hosted on GitHub. This is the canonical location for all source files, commit history, branch management, and issue tracking.

- Repository URL: <https://github.com/sanikadeshmukh3/Habitat>
- Issue Tracker: <https://github.com/sanikadeshmukh3/Habitat/issues>

1.1 Cloning the Repository

```
git clone https://github.com/sanikadeshmukh3/Habitat.git
cd Habitat
```

1.2 Viewing File Change Logs

All significant changes include descriptive commit messages. A developer reviewing why code evolved in a certain way should consult the commit history via the GitHub Commits tab or the command line.

```
# Full commit history
git log

# Changes to a specific file
git log --follow -- path/to/file

# Diff for a specific commit
git show <commit-hash>
```

Commit history is also browsable at:

<https://github.com/sanikadeshmukh3/Habitat/commits/main>

2. Prerequisites

Ensure the following are installed before building the project. No other global tools are required — all other dependencies are managed via npm.

- **Node.js v18 or higher**
- **npm** (bundled with Node.js)
- **Expo Go** — installed on a mobile device, or an iOS Simulator (macOS only) or Android Emulator running before starting the app
- **PostgreSQL v14 or higher** — required for local backend development and the DB-connected integration tests. Not required if using the live deployment.

To verify Node.js and npm:

```
node -v  
npm -v
```

Both should return a version number. If not, install Node.js from <https://nodejs.org>.

To verify PostgreSQL (local development only):

```
psql --version
```

3. Building & Running from Source

3.1 Environment Configuration

The backend requires environment variables for database connectivity, authentication, email delivery, and AI services. Create a `.env` file in the `backend/` directory:

```
DATABASE_URL=<your-postgresql-connection-string>  
JWT_SECRET=<your-jwt-secret>  
OPENAI_API_KEY=<your-openai-key>  
SENDGRID_API_KEY=<your-sendgrid-key>
```

Note: Evaluators using the existing deployment do not need to configure this — the live backend at <https://habitat-qel9.onrender.com> is already configured. Local development requires your own PostgreSQL instance with the schema applied (see section 3.3).

3.2 Running the Full Application (Recommended)

From the root of the repository, a single setup script installs all dependencies and starts both the backend server and the mobile application:

```
./setup.sh
```

- Once Expo starts, press **a** to launch on Android emulator or **i** for iOS simulator
- Ensure your emulator or simulator is running before pressing **a** or **i**

3.3 Running Components Separately

Backend

```
cd backend
npm install
npx prisma migrate deploy # applies schema to your local PostgreSQL database
npm start
```

Mobile

```
cd mobile
npm install
npx expo start
```

3.4 Database Seeding (Optional)

A seed script populates a demo user pre-configured for the habit stacking feature, with 35 days of check-in history and a proving window already elapsed. This is useful for demonstrating the stacking activation flow without waiting for real time to pass.

```
cd backend
node prisma/seed-stacking-demo.js

# Demo login credentials:
# Email: demo@habitat.com
# Password: password123
```

4. Issue Tracking

Habitat uses GitHub Issues for bug tracking, feature requests, and development tasks across all team members.

- Issue tracker: <https://github.com/sanikadeshmukh3/Habitat/issues>
- **Open tab:** active bugs and in-progress tasks
- **Closed tab:** resolved issues with associated fixes and context

Issues span both backend and mobile development and are assigned to the responsible team member. Known open issues at Release 1.0 are noted in the Known Failures section of the test documentation (TESTING.md).

5. Architecture and Design Patterns

Habitat follows an MVC architecture. The backend separates concerns into routes (request routing), controllers (HTTP handling), and services (business logic), with Prisma as the data access layer. The mobile frontend uses React Native with Expo, organized around screen components, custom hooks, and a centralized API client.

Design patterns are identified with inline comments at the point of use in the source files. The following are the primary patterns in the Release 1.0 codebase.

5.1 Singleton — React Query QueryClient

File: [mobile/lib/queryClient.ts](#)

React Query's [QueryClient](#) is instantiated once and shared across the entire mobile application via a [QueryClientProvider](#) in [mobile/app/_layout.tsx](#). This ensures all components share a single cache, preventing redundant network requests and keeping UI state consistent.

5.2 Strategy — Tier Assignment

File: [backend/services/tierService.js](#)

The habit stacking system assigns one of three tiers (TIER_1, TIER_2, TIER_3) to each habit based on its consistency score. The [assignTier](#) function encapsulates the scoring policy as a swappable strategy — the stacking pipeline delegates to it without knowing the threshold logic. This means thresholds can be changed in one place without touching any caller.

5.3 Repository — Prisma Service Layer

Files: `backend/services/checkinService.js`, `backend/services/enrollmentService.js`, `backend/services/provingWindowService.js`, and related service files

All database access is abstracted behind service modules rather than being written directly in controllers. Controllers call named service functions; they never write raw Prisma queries. This Repository pattern means the data access layer is independently testable — in the unit tests, service modules are replaced with `jest.fn()` stubs without touching controller logic.

5.4 Observer — React Query Cache Invalidation

Files: `mobile/hooks/useCheckIn.ts`, `mobile/hooks/useHabits.ts`

When a check-in is submitted, the mutation hook calls `queryClient.invalidateQueries(['habits'])` and `queryClient.invalidateQueries(['checkins'])`, which notifies all subscribed components to refetch. This implements the Observer pattern: components subscribe to a query key and react to invalidation without the mutation needing to know which components are listening. The comment `// Design pattern: Observer` marks the invalidation calls.

5.5 Pure Functions — Streak and Badge Logic

Files: `backend/lib/streaks.js`, `backend/lib/badges.js`

The streak calculation library and badge evaluator are written entirely as pure functions with no side effects and no external dependencies. This functional design principle makes the gamification logic fully deterministic and independently testable without any mocking. All eight streak functions and all ten badge definitions are exercised in the unit test suite with boundary tests at every threshold. A comment at the top of each file notes this design principle.

6. What Is New in Release 1.0

Release 1.0 is a substantial expansion over Beta. A developer picking up the project should be aware of the following major additions:

- **Habit stacking:** A complete progressive onboarding system. Habits with low consistency scores trigger an enrollment flow. Enrolled users see a dashboard status card, a ranked habit queue with drag-to-reorder, and activation/snooze modals when proving windows elapse. Implemented across six backend services (`enrollmentService`, `activationService`, `provingWindowService`, `consistencyService`, `tierService`, `stackingTriggerService`) and six frontend screens.

- **Calendar overhaul:** Habits are scoped to their lifetime — no dots appear before the creation date or on future dates. Retroactive check-ins are blocked. Seven distinct visual states distinguish live vs. deleted habits and the presence or absence of feedback on each check-in.
 - **Soft deletion:** Deleting a habit sets `active=false` and records `deletedAt` rather than destroying the row. Historical check-in data is preserved and remains visible in the calendar in a distinct visual state.
 - **Edit habit screen:** Users can rename and recategorize habits. Duplicate name detection is enforced case-insensitively. The frequency toggle is disabled after creation to prevent streak data inconsistencies.
 - **Weekly habit check-in locking:** Check-ins made on a prior day in the same ISO week are locked and cannot be undone. Same-day check-ins remain editable.
 - **Wrapped weekly summary:** An AI-generated weekly recap with animal archetypes, completion scores, and a refreshable summary backed by OpenAI.
 - **Bug fixes:** 35-day dot grid formula corrected (`backend/controllers/habit-controller.js`), weekly streak double-increment prevented, routing flow after AI habit creation fixed, inaccurate metric calculations corrected.
-

7. Running Tests

Full test documentation — including all manual UI flows, coverage data, and the complete 1.0 Release results — is in `Habitat_TestSuiteDocumentation`.